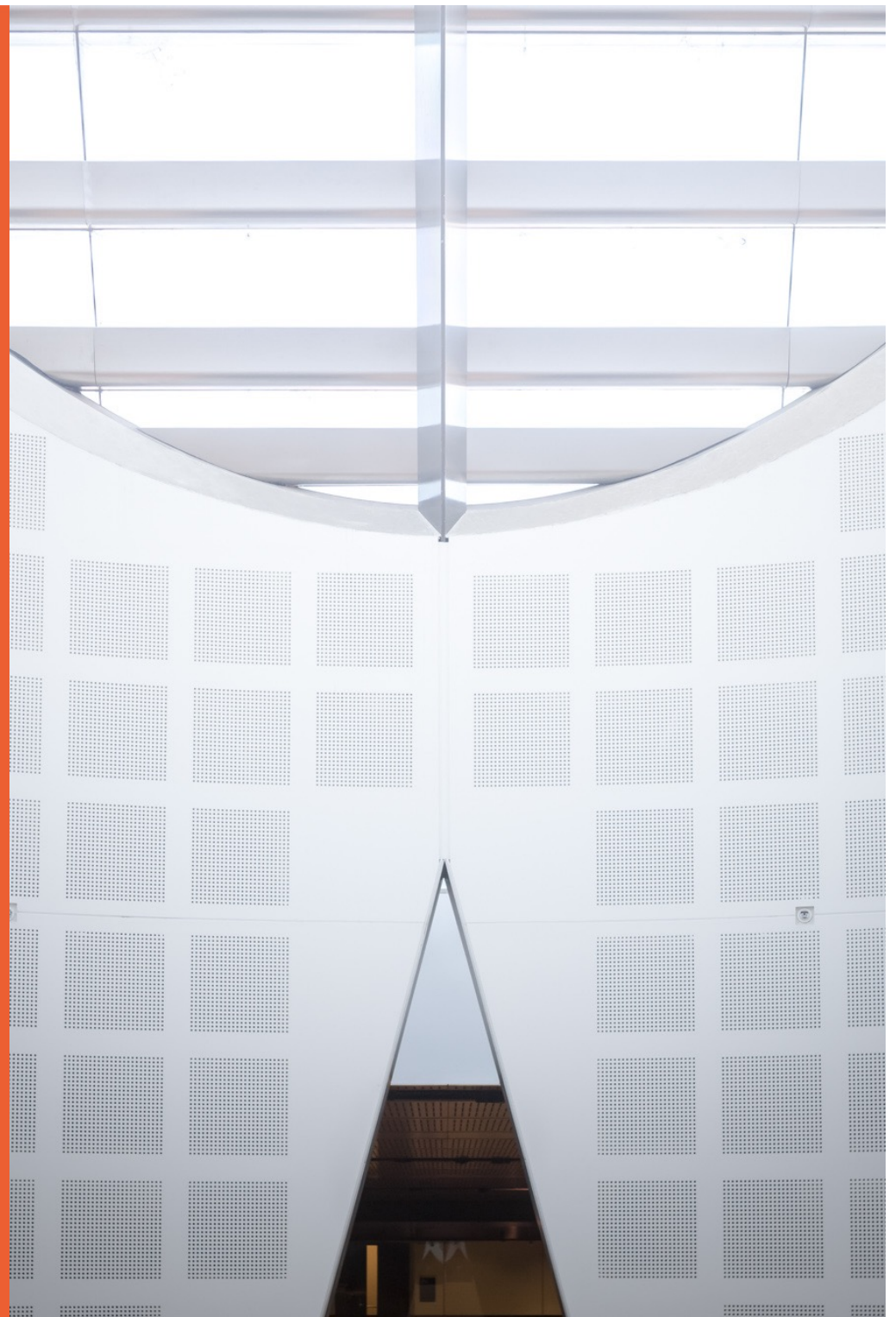


Linux, Networking and the Cloud

Presented by
Dr. Huaming Chen

ELEC3609/9609
Internet Software Platforms
2025 S2



We acknowledge the tradition of
custodianship and law of the Country on which
the University of Sydney campuses stand.
We pay our respects to those who have cared
and continue to care for Country.



THE UNIVERSITY OF
SYDNEY

Overview

✓ Linux

- ✓ Linux Users
- ✓ Capabilities
- ✓ SSH
- ✓ Package Managers

✓ Amazon Web Services

- ✓ EC2
- ✓ EBS
- ✓ VPC
- ✓ RDS
- ✓ Route 53
- ✓ EKS/ECS

✓ Ports & Firewalls

- ✓ TCP Ports
- ✓ Firewalls
- ✓ Security Groups
- ✓ Elastic Load Balancer
- ✓ Elastic IPs

✓ Server Applications

- ✓ Web Servers
- ✓ Application Interface
- ✓ Databases
 - ✓ Volume Separation

✓ Lab Schedule

Linux

Introduction

- Linux is an open-source UNIX-like operating system, whose defining feature is the Linux kernel. Released in 1991 by Linus Torvalds.
- A Linux distribution defines a collection of software that comes pre-packaged as a complete operating system.
- For example:
 - package managers (`apt-get` or `pacman`)
 - window managers (GNOME or `i3`)
 - text editors
- Linux has become the most widely installed general purpose OS due to the proliferation of Android. It is also the most dominant OS for web servers (>90%).

Linux Users

```
cat /usr/include/bits/typesizes.h | grep UID_T
```

```
#define __UID_T_TYPE __U32_TYPE
```

- The 'root' user (aka the `superuser`) is the most privileged user on a Linux system.
- While the root user could have a wider variety of privileges.
- **Programs execute as a particular user.**
- If a program requires special privileges on the system, it may need to execute as the root user.
- The output of the command `ps aux` will show you a list of all processes running on the system and the user they are running as.

Passwd

Users are listed and managed by the files `/etc/passwd` and `/etc/shadow`.

You should read and understand the details of these files on the [Wikipedia Passwd page](#).

Program-Specific Linux Users

- Usually, a program will not need root access to the system, and so will (or at least, it should) run as a different user.
- If such a program is somehow compromised, it will only be able to perform malicious actions that its running user could perform.
- For example:
 - Web servers (including `nginx`) usually run as the `www-data` user.
 - MySQL programs (including `mysqld`) usually run as the `mysql` user.
- Linux systems often come with a `nobody` user, which can run programs that require almost no special privileges.

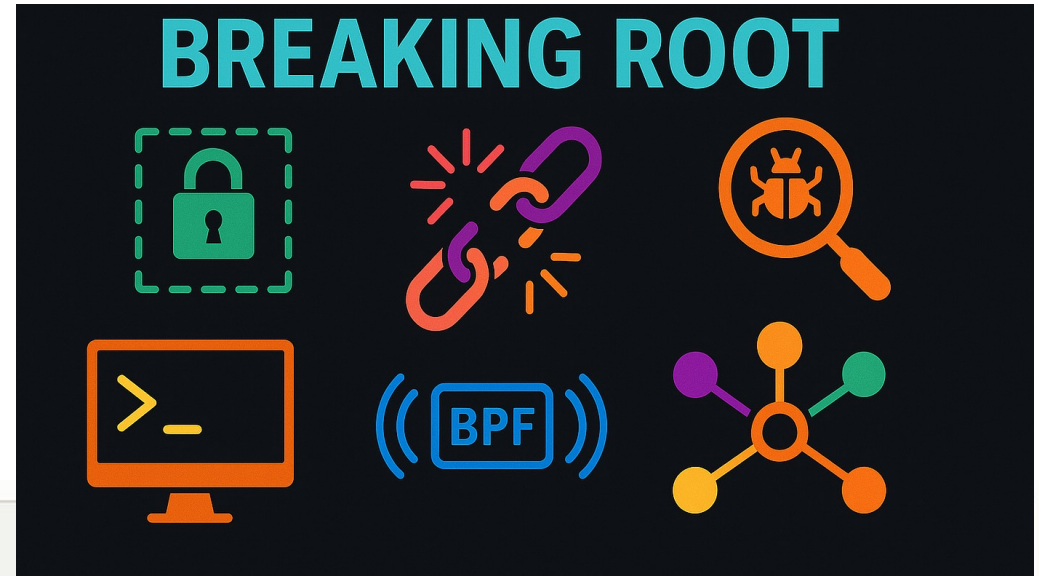
Capabilities

- Traditional root vs. Capabilities
 - In a classic UNIX/Linux model:
 - root (UID 0) has all privileges.
 - If a process is compromised while running as root, the attacker gains full control of the system.
 - Capabilities split root privileges into smaller, fine-grained units (like `CAP_NET_BIND_SERVICE`, `CAP_SYS_ADMIN`, `CAP_DAC_OVERRIDE`).
 - Each process can be assigned only the subset of privileges it really needs.
- Why it improves security
 - Least privilege principle
 - Damage containment
 - Reduced attack surface

Capabilities

- Running **nginx**:
 - Traditionally: needed root just to bind port 80.
 - With capabilities: run as unprivileged user but grant only `CAP_NET_BIND_SERVICE`.
 - If nginx is hacked, the attacker cannot:
 - Create raw sockets (`CAP_NET_RAW`).
 - Mount filesystems (`CAP_SYS_ADMIN`).
 - Change ownership of files (`CAP_CHOWN`).

Capabilities



Set Name	Description
Effective	Capabilities currently being used for permission checks
Permitted	Capabilities that can be moved to the effective set
Inheritable	Capabilities that can be inherited by child processes
Bounding	Maximum capabilities a process can ever receive
Ambient	Capabilities passed to non-SUID files without defined capabilities

Capabilities

```
nick@phoenix:~$ ll /bin/ping
-rwsr-xr-x 1 root root 64424 Jun 28 19:05 /bin/ping*
nick@phoenix:~$ ll /usr/bin/passwd
-rwsr-xr-x 1 root root 59640 Mar 23 03:05 /usr/bin/passwd*
```

```
nick@phoenix:~$ sudo chmod 755 /bin/ping
nick@phoenix:~$ ll /bin/ping
-rwxr-xr-x 1 root root 64424 Jun 28 19:05 /bin/ping*
nick@phoenix:~$ ping baidu.com
ping: socket: Operation not permitted
```

```
nick@phoenix:~$ sudo setcap cap_net_admin,cap_net_raw+ep /bin/ping
nick@phoenix:~$ getcap /bin/ping
/bin/ping = cap_net_admin,cap_net_raw+ep
nick@phoenix:~$ ping baidu.com
PING baidu.com (39.156.69.79) 56(84) bytes of data.
64 bytes from 39.156.69.79 (39.156.69.79): icmp_seq=1 ttl=128 time=33.9 ms
64 bytes from 39.156.69.79 (39.156.69.79): icmp_seq=2 ttl=128 time=34.6 ms
```

Installing Linux

- We usually install Linux from a CD image or ISO file.
- It's usually best to Google for a local mirror from which to download the ISO for the distribution you want.

Downloading Ubuntu

For example, Ubuntu ISOs can be downloaded from:

mirror.aarnet.edu.au/pub/ubuntu/releases

- To try things out locally, you can try installing VirtualBox and inserting the Linux ISO into the virtual machine's CD-ROM drive.

Linux From Images

- Depending on your setup, you can often download or use pre-existing Linux images.
 - Docker
 - hub.docker.com/_/ubuntu
 - VirtualBox
 - virtualboxes.org/images/ubuntu
 - Amazon Marketplace
 - aws.amazon.com/marketplace
 - Vagrant
 - atlas.vagrantup.com/boxes/search

Secure Shell (SSH)

- Secure Shell (SSH) is a secure replacement for the old Telnet protocol.
- SSH enables a user to access the shell or terminal on a remote machine.
- Essential tool for any web master, since Linux is the dominant server platform on the internet.

“SSH to it”

To “SSH” to a machine means to use an SSH client on your local machine, to connect to the remote server’s SSH server, providing you with a shell on that server.

SSH Server

- The most common SSH server software is OpenSSH, which originates from the OpenBSD project.
- An SSH server:
 - Listens for incoming connections on TCP Port 22.
 - Authenticates connecting users using:
 - Passwords
 - Public/Private Keys

SSH Servers

While OpenSSH often comes pre-installed on many Linux distributions, there are many options available:

en.wikipedia.org/wiki/Comparison_of_SSH_servers

SSH Client

- Open SSH also provides an SSH client, which is the client-side software that connects to the SSH server.
- There are many alternative clients, depending on the platform you are using.
- On Linux, you're best to stick with the default OpenSSH.
- On Windows however, PuTTY might be a better option.

SSH Clients

There are many SSH Clients to choose from.

en.wikipedia.org/wiki/Comparison_of_SSH_clients

SSH Keys

- SSH provides *authentication* and *encryption*.
- The obvious way to log in to SSH is using the password of the remote user. This is bad.
- Instead, we want to use SSH keys.
- There are two *sets* of keys that SSH uses:
 - **Server:** The server has its own key pair, which allows initial connections to it to be encrypted with its public key. It also allows clients to authenticate the server.
 - Only the server possessing the private key can read incoming connections.
 - **Client:** Each client has their own key pairs, which are used to authenticate when connecting to the server.

Public Key Crypto & Signatures

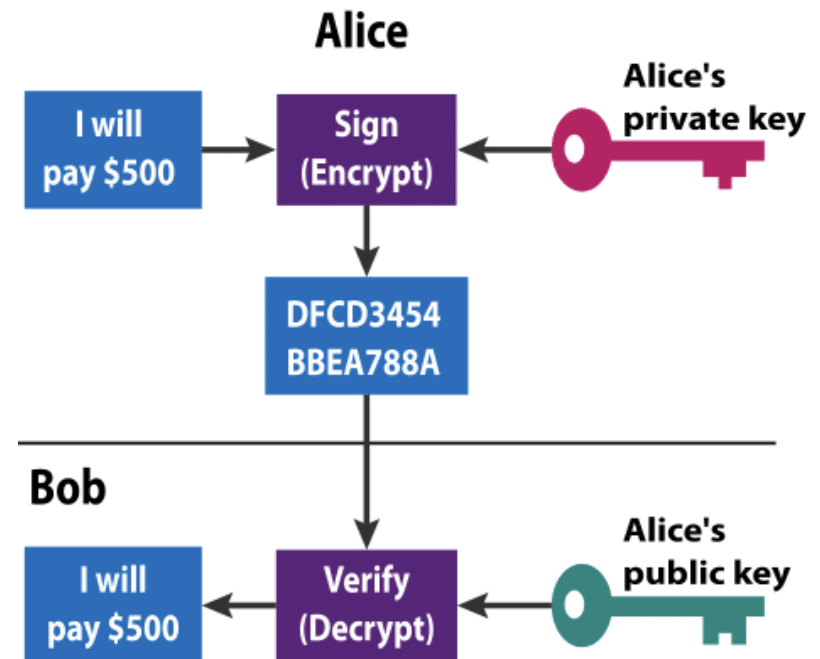
- An encryption and decryption key pair are generated to provide effective authentication. This is achieved by demonstrating possession of the private key without revealing it.

- Private Key

- Held by you, never shared.
- Only you can sign a request with your private key.

- Public Key

- Shared publicly.
- Anyone can verify your signature with your public key.



SSH Server Keys

- The server typically stores its key pairs as:
 - `/etc/ssh/ssh_host_rsa_key`
 - `/etc/ssh/ssh_host_rsa_key.pub`
- When first connecting to a server that you have never connected to before, you will get a warning like this:

SSH to new server

```
The authenticity of host '123.123.123.123' can't be established.  
ECDSA key fingerprint is  
SHA256:eJ0v0MmL9P2ANc00Ltowb3iKbAaMHYe/c4LMeg39pHE.  
Are you sure you want to continue connecting (yes/no)?
```

- By accepting this, a new entry is added to your `~/.ssh/known_hosts` file.

SSH Server Keys – `known_hosts`

- By keeping track of known SSH server public keys, SSH clients can minimise the potential for a MITM attack.

Host Key Change

If the key changes for a known SSH server, the SSH client won't permit the connection, as it could be a MITM attack.

If you get this warning, you should confirm that the key was actually changed before connecting.

SSH Client Keys

- To authenticate a particular user on the SSH server, a client proves ownership of a private key, which corresponds to an authorised public key.
- For example, say we want to connect to a server as the user `myuser`.
 - On the server, there is a file in `myuser`'s home directory:
 - `/home/myuser/.ssh/authorized_keys`
 - This file contains the public keys of anyone who is authorised to connect to the server as this user.
 - The SSH server will check that any client trying to connect possesses the private key corresponding to a public key in that file. If they do, then the connection is authorised.

SSH Agent

- `ssh-agent` is a useful program for storing the unencrypted SSH private key in memory.
- SSH typically reads private keys from `~/.ssh/id_rsa`, which is encrypted. Every time SSH is run, it needs the password to use the key.
- To make things easier, the unencrypted version can be stored in memory by attaching it to a running process.
- Since we don't want to store it in every SSH client process, we run `ssh-agent` to store it for us.

SSH Agent Forwarding

- `ssh-agent` runs on the client, and as new SSH processes connect to it, they ask it to perform signing operations using the private key.
- With the `ForwardAgent` option, we can allow remote SSH clients to access our local `ssh-agent` process (and therefore, our unencrypted private key).

Example `~/.ssh/config`

```
Host example
    HostName 52.61.143.178
    User ubuntu
    IdentityFile ~/.keys/some-key.pem
    ForwardAgent yes
```

Terminal Text Editors

- Previously, we spoke about text editors like Atom or Notepad++ to use on our desktop.
- However, since these are graphical applications, we cannot use them on a remote server via SSH.
- Instead, we need to use a command-line text editor.
 - Typically, you'll use this to edit configuration files on the remote server.

Terminal Text Editors

- Many options, with lots of customisability:
 - Nano
 - Very simple, and often the default on many versions of Linux.
 - Familiar editor interface, using arrow keys to navigate.
 - Exit the program using Ctrl+X or write using Ctrl+O.
 - Vim
 - Very flexible, fully-featured text editor. Can be challenging to learn.
 - Edit the file by entering insert mode with ‘i’.
 - Exit a particular mode using ‘esc’.
 - Exit the program using ‘:’ to access commands, then ‘q’ to quit or ‘wq’ to write your changes and quit.

Package Managers

- Ubuntu's package manager is Advanced Packaging Tool (APT).
- It is easiest to install software on servers using the package manager, since it can automatically update the software for us.
- However, we may need to install the software manually if we can't use the package manager:
 - There is no package for the software we want.
 - The package software is out of date.
 - We want to install into a different location.
 - The software must be compiled with different options than the package manager provides.
- More information can be found [here](#).

Amazon Web Services (AWS)

Amazon Web Services

- Amazon Web Services (AWS) is an extremely popular cloud hosting provider.
- Using AWS removes the need to set up dedicated hardware to host a website, and often takes care of software installation and configuration.
- AWS provides a wide range of services that can assist in the creation of website infrastructure.



Elastic Compute Cloud (EC2)



- Provides access to server instances.
- EC2 is the most common and fundamental of the AWS suite, and you will need to use it for deployment of your project.
- EC2 instances can be created in one of many availability zones (AZs) located in various regions of the world.

Watch The Video

Watch the introduction video on the link below, to make sure you understand what EC2 offers.

aws.amazon.com/ec2/

Elastic Block Store (EBS)



- Provides hard drive volumes that can be attached to EC2 instances.
- Useful for separating your data storage from the server that executes your software.
- When attaching an EBS volume to an EC2 instance, both the volume and instance must be in the **same Availability Zone**.

Watch The Video

Watch the introduction video on the link below, to make sure you understand what EBS offers.

www.youtube.com/watch?v=DKftR47Ljvw

Relational Database Service (RDS)

- Provides managed database servers.
- Useful for having a scalable, backed-up version of your data.
- Supports: Aurora, PostgreSQL, MySQL, MariaDB, Oracle Database and SQL Server.
- Video: [Understanding Amazon Relational Database Service \(RDS\)](#)



Route 53

- Provides the ability to route requests to AWS infrastructure.
- [www.example.com](#) => 53.96.54.23
- Operates a Domain Name System (DNS) web service.
- Video: [Amazon Route 53](#)



Virtual Private Cloud (VPC)

- Provides support for network segregation and overall greater control over EC2 instances.
- Very useful tool for web infrastructure, but its configuration can be complex.
- **Not necessary for the project** – instead, we will use the default VPC created by AWS.
 - EC2-VPC: Using a VPC within EC2.
 - EC2-Classic: Not using a VPC.



Amazon Elastic Kubernetes Service (EKS)

- Provides a managed Kubernetes service.
- Maintains the availability and scalability of the Kubernetes control plane.
- Integrates with other AWS technologies

- Video: [Deep Dive on Amazon EKS](#)



Amazon EKS

Amazon Elastic Container Service (ECS)

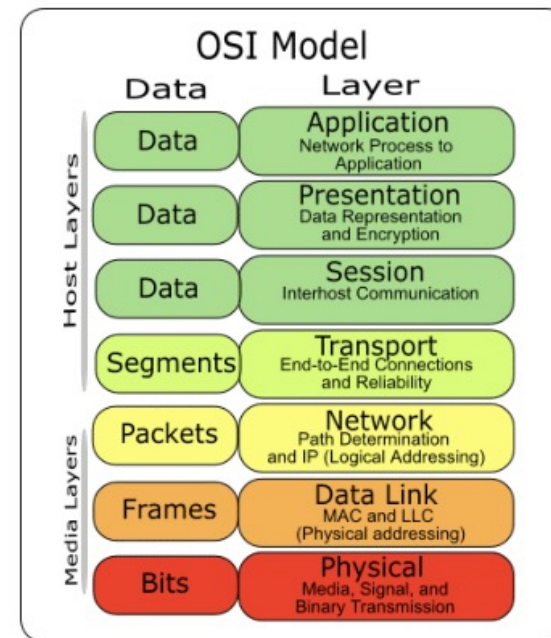
- Provides a managed container orchestration service.
- Amazon's solution to Kubernetes.
- Integrates with other AWS technologies
- Video: [Amazon ECS: Core Concepts](#)



Ports & Firewalls

Network Revision: OSI Model

- The OSI Model is a way to conceptualize network systems, with each layer being an abstraction of the previous.
- To properly understand interactions between technologies, you should be quite familiar with this model.



The OSI Model - [Wikimedia](#)

Ports & Firewalls

- All of the software we are running involves responding to network requests.
- Because of this, it is necessary for us to understand networking concepts and their operation.
- In particular:
 - TCP/UDP Ports
 - OSI Model – Layer 4
 - Firewalls
 - Load Balancers
 - Public IPs
 - OSI Model – Layer 3

TCP Ports

- Most web software communicates via the Transmission Control Protocol (TCP).
- TCP establishes an end-to-end connection and ensures delivery of packets in the order they were sent.
- We say that applications ‘listen’ on a particular TCP port.
 - This allows multiple applications to run on the same host, with each listening for TCP connections on a different port.
 - Any application can choose to listen on any port, but there are standard port numbers used for particular application types.

Common Ports

You *must* know these common TCP ports:

80: HTTP

443: HTTPS

22: SSH

TCP Ports - Listening

- Whenever a network service starts on a host, it attempts to listen on a particular port.
- Only one running application may listen to a port at a time.

Privileged Ports

On Linux, port numbers below 1024 are “privileged ports”.

Applications require root access if they wish to listen on these ports.

netstat

Lets take a look at what is listening on our host:

```
netstat -nlpW
```

Take a look at what each of the flags to netstat do.

UDP Ports

- The User Datagram Protocol (UDP) is a connectionless protocol.
- It is a 'best-effort' protocol, and makes no guarantees about packets being in order, or even being delivered at all.
- UDP has its own set of ports.
 - For example, TCP Port 80 is different from UDP Port 80.

Common Ports

You *must* know this common UDP port:

53: DNS

Firewalls

- A firewall is a security system that can allow or disallow network traffic to pass according to a set of rules.
- Firewalls come in many shapes and sizes:
 - Your Home Router
 - Makes decisions about incoming/outgoing traffic and prevents attackers from gaining access to your network.
 - Software Firewalls
 - Runs on a host machine and provides rules about the traffic allowed to enter the host.

UncomplicatedFirewall (ufw)

Ubuntu comes with the software firewall `ufw` installed by default. Sometimes, it can get in the way if not configured properly. Be aware of it when trying to resolve networking issues.

AWS Security Groups

- AWS Security Groups (SGs) are sets of firewall rules that can be applied to EC2 instances.
- When first configuring an EC2 instance, you can either create a new security group or add the instance to an existing security group.
- To configure a security group, you must specify:
 - Protocol: TCP/UDP/ICMP
 - Port(s): The TCP/UDP port(s) the rule applies to.
 - Source: Which source IP address(es) the rule applies to.
- The rules are a whitelist, meaning any packets not matching a rule are automatically dropped (disallowed).
- For development, you may find it useful to add a rule for SSH access from any source (0 . 0 . 0 . 0 / 0)

Elastic Load Balance (ELB)



- Provides a load balancing service for EC2 instances.
- Useful tool for production websites, allowing for easy horizontal scaling.
- For your project, you should learn how to utilize ELB, configuring it for your deployment.

Watch The Video

Watch the introduction video on the link below, to make sure you understand what ELB does.

~~www.youtube.com/watch?v=oEcEqN8PecI~~

Elastic IPs

- When you start an EC2 instance, AWS assigns it a public IP address.
- If you restart the instance, it can be reassigned to a different IP address, which can make updating dependent configurations very difficult.
 - For example, updating the DNS records.
- To maintain a consistent IP address between restarts, you can assign the instance an Elastic IP address.
 - Once assigned, the instance will always have this IP whenever it is created

Server Applications

Server Applications

- You will need to configure several applications on your server in order to deploy a website properly.
- In particular, you should install:
 - A web server.
 - An application interface.
 - The web application.
 - A database.

Web Servers

- A web server receives incoming HTTP requests, processes them, and usually provides a HTTP response.
- There are many types of web server software, including:
 - Apache
 - Nginx
 - IIS
- Specific details on each of these can be found in the Week 3 Lecture.

Recommended Web Server Software: nginx

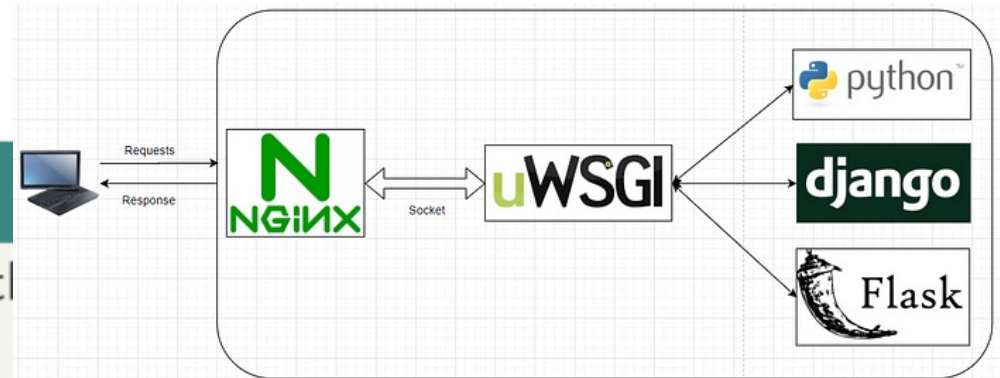
- `nginx` (pronounced ‘engine-X’) is a fast web server that can also act as a reverse proxy.
- Due to its widespread use and ease of configuration, we highly recommend you use `nginx` for your project.
- While `nginx` can receive and process a HTTP request, it does not interpret web application code.
- Instead, we ask `nginx` to `proxy_pass` HTTP requests to the application interface.

Application Interfaces

- A web application interface provides a simple interface between a web server and the application server.
- It runs multiple application ‘workers’ so that the application can process requests concurrently.
- WSGI is a specification for such an interface, and it is implemented by the uWSGI software.
- uWSGI is designed for Python programs, and has good support for Django.

Django + uWSGI + nginx

Work through this tutorial to get t



uwsgi-docs.readthedocs.io/en/latest/tutorials/Django_and_nginx.html

Databases

- Most web applications require persistent storage of data.
- You will most likely need to install a relational database management system (RDBMS) for your project.
- We recommend using MySQL, since it is feature-rich and relatively easy to use.
 - PostgreSQL and MariaDB are acceptable alternatives.
- Regardless of your choice of RDBMS, the rough setup steps will be:
 - Install the DB software.
 - Configure users (username & password).
 - Configure application code to use the DB.
 - Connect using the username & password.

Separation of Data

- You should always separate your data from your application code.
- Typically, this involves configuring your DB to save data to a separate volume from where your operating system and web application are installed.
- If working on EC2, you should utilise a separate EBS volume for database storage.
 - <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-attaching-volume.html>
- With this, you can configure your RDBMS to store data on the new volume.
 - For MySQL:
 - <https://www.digitalocean.com/community/tutorials/how-to-move-a-mysql-data-directory-to-a-new-location-on-ubuntu-16-04>

Your Task

**You reached the end of the lecture.
Thank you and see you next week.**